Circuit Breaker

stop calling what's certain to fail

Watch the 4-min video

In one line: a circuit breaker wraps calls to one dependency and, once failures cross a threshold, **opens** so calls fail instantly with a fallback; after a cool-down it goes **half-open** and lets a few trial calls decide whether to close again or stay open.

1 Waiting for a call that will fail

The recommendation service is down. Every product page still calls it, waits the full 2 s timeout, and then fails. Thousands of requests a second are spent on calls that are certain to fail, threads pile up behind the timeouts (lesson 4), and the struggling service gets no chance to recover because it is still being hammered. A circuit breaker, like the one in your house, cuts the circuit.

2 Three states

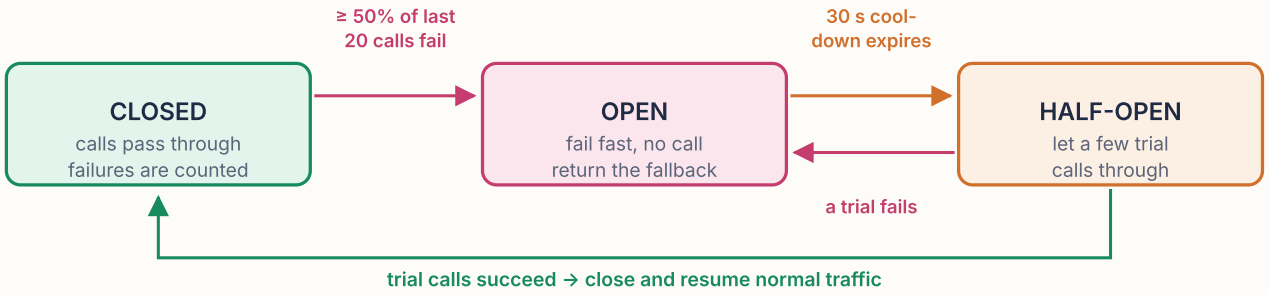


Figure 1. Closed → open on too many failures; open → half-open after a cool-down; half-open closes on success or reopens on failure.

- **Closed** (normal): calls go through; the breaker records each result in a sliding window.
- **Open**: once the failure rate crosses the threshold (e.g. 50% of the last 20 calls, or 5 consecutive failures), calls fail immediately without touching the dependency.
- **Half-open**: when the cool-down (e.g. 30 s) expires, a small number of trial calls are let through. If they succeed the breaker closes; if any fails it opens for another cool-down.

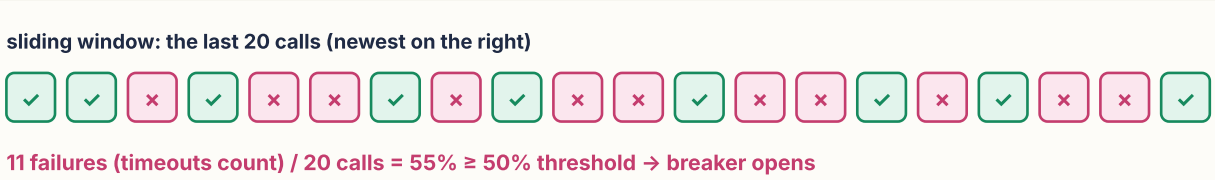


Figure 2. A count-based window. Real libraries also need a minimum number of calls before they judge, so 1 failure out of 1 call doesn't open the breaker.

Libraries name these settings explicitly. In **Resilience4j** (Java) they are `failureRateThreshold`, `slidingWindowSize`, `minimumNumberOfCalls`, `waitDurationInOpenState` and `permittedNumberOfCallsInHalfOpenState`; **Polly** (.NET) and **pybreaker** (Python) have equivalents, and service meshes such as Envoy/Istio offer outlier detection that ejects failing hosts (lesson 27).

3 Fallbacks, and how it differs from retries

When the breaker is open the caller needs an answer: **cached data**, a **default value**, an empty list, or a clear error the UI handles well (lesson 8). Failing in a millisecond keeps threads free and latency low.

	Retry	Circuit breaker
Assumes the failure is	brief: try again soon	lasting: stop trying for a while
Effect on the dependency	adds load	removes load, lets it recover
Scope	one request	all requests to that dependency
Together	retry 1–2 times inside the breaker; when the breaker is open, no retries happen at all (lesson 5)	

WORKED EXAMPLE: A FAILING RECOMMENDATION SERVICE

Recommendation calls start timing out. Once **50% of the last 20 calls** fail, the breaker opens. Product pages immediately show a **default list of best sellers** instead of waiting 2 s per request. After a **30 s** cool-down the breaker goes half-open and lets a few trial calls through; they succeed, the breaker closes, and personalised recommendations return. A metric and log line on every state change tells the team exactly when it opened and closed.

COMMON MISTAKES

- One breaker shared by several dependencies, so one failure blocks calls to healthy ones. Use one per dependency, sometimes per endpoint.
- Not counting timeouts as failures, so a hanging dependency never trips it.
- Treating 4xx client errors as failures, so bad input from one user opens the breaker.
- Writing your own instead of using a tested library, and emitting no metric when it opens.

4 Key terms

Closed: normal state: calls pass, results counted

Half-open: after the cool-down, a few trial calls probe recovery

Cool-down: how long the breaker stays open before trying again

Open: calls fail fast without reaching the dependency

Failure threshold: failure rate or count that opens the breaker

Fallback: what to return while the breaker is open

5 Interview questions

Q1. Explain the circuit breaker states. BASIC

Closed passes calls and counts failures; open fails fast once a threshold (e.g. 50% of the last 20 calls) is crossed; half-open lets trial calls through after a cool-down to decide whether to close or reopen.

Q2. What does the caller return when the breaker is open? BASIC

A fallback: cached or default data, an empty list, or a clear error that the UI handles gracefully.

Q3. How is a circuit breaker different from retries? SCENARIO

Retries assume a brief failure and try again soon; a breaker assumes a lasting failure and stops calling for a while. Use both: a couple of retries inside a breaker that stops all calls when failures persist.

Q4. One service calls payments, search and reviews. How many breakers? SCENARIO

At least three, one per dependency, so a reviews outage doesn't block payments; sometimes one per endpoint if they fail independently.

Q5. Why is a half-open state needed at all? TRICKY

Without it you'd either never re-close or send full traffic to a dependency that may still be broken; a few trial calls test recovery cheaply.

Q6. Could a breaker hurt availability? TRICKY

Yes, if it's too sensitive (no minimum call count, counting 4xx) it opens on noise and turns a healthy dependency into fallback-only. Tune thresholds and monitor state changes.

Next: SD-K-007 · Bulkhead Pattern backendroadmap.com